Executable Self-Justifying Axiom Systems in Proflog: A System Description

Author information pending

Affiliation pending

Abstract. Willard-style self-justifying axiom systems (SJAS) identify weak arithmetic settings in which an axiom system can consistently state selected forms of its own consistency. The core tradeoff is operationally suggestive: the object language keeps enough arithmetic to code syntax and proofs, while refusing the strength that comes with total multiplication. This system description reports on an implementation of a finite ordinary-tableau SJAS substrate inside Proflog, a tableau-based logic programming system. The implementation generates a finite $IS\#_D(\beta)$-style axiom basis, gives formulas, systems, and proof certificates inspectable object-language codes, and checks proof predicates through Proflog's first-order/equality tableau kernel. The result is both a mechanization of a metamathematical construction and a small executable language experiment: user clauses can be reflected into the system being justified, or kept external as ordinary Proflog procedures, and the distinction changes which proofs the internal proof predicate may cite.

Keywords: self-justifying axiom systems · logic programming · semantic tableaux · proof checking · declarative programming

1 Introduction

Willard's self-verifying and self-justifying axiom systems are boundary-case exceptions to broad forms of Gödel's second incompleteness theorem. They do not make strong arithmetic prove its own consistency. Instead, they weaken the base language so that selected self-consistency statements can be added without destroying consistency [3, 5]. In the Type-A line most relevant to ordinary semantic tableaux, addition is available as total arithmetic while multiplication is represented by a three-place graph relation rather than by a total function symbol.

This paper reports on an implementation of that idea in Proflog. Proflog is Fitting's tableau-based alternative to Prolog: programs are first-order formulae with equality, evaluated by tableau closure plus a Procedure Call rule [1]. That makes it a useful host for SJAS work. The same kernel that executes a Proflog program can also serve as the deduction method D whose proofs are coded and inspected by the object language.

The contribution is not a new proof of Willard's metatheorems. It is an executable account of what must be mechanized before an SJAS becomes a programming substrate:

- a finite reflected axiom basis with generated Group-Zero through Group-3 records;
- a Proflog proof profile for binary U-grounding arithmetic, formula-code predicates, and proof-code predicates;
- inspectable formula, system, and proof encodings rather than hash labels;
- object-language proof predicates that validate certificates through the core tableau kernel;
- examples distinguishing reflected user clauses, which become citeable Group-2b axioms, from external clauses, which remain executable but are not part of the self-consistency claim.

2 System Availability and Scope

The working system is available as part of Proflog at `https://github.com/autarkenterprises/proflog`. The implementation described here corresponds to the main-line SJAS work through commit `14d3150`; the focused user-facing examples are in `worked-examples/willard-sjas.md`, and the regression suite is `test/proflog/willard_sjas_test.clj`. The dedicated commands are:

```
lein test-proflog-sjas
lein test-proflog-sjas-slow
```

This is a system description of a working profile, not a claim that every Willard variant has been implemented. The implemented deduction method D is Proflog's ordinary semantic-tableau kernel. The profile deliberately does not yet include Tab-1 theorem reuse or proof-list checking.

3 Background

3.1 Proflog

Fitting's Proflog replaces resolution over Horn clauses with semantic tableaux and procedure calls [1]. A query succeeds when the tableau for its negation closes, fails when the tableau for the query itself closes, and remains unresolved when neither direction closes in the admitted search. The implementation used here is written in Clojure's `core.logic` [2]. Its kernel records explicit proof terms for ordinary tableau steps, equality and disequality closure, and positive and negative procedure calls.

3.2 Willard SJAS

Willard's systems combine a weak arithmetic base with a reflected statement about the consistency of a deduction method. The construction is sensitive to which arithmetic functions are admitted as total. In the Type-A tableau line, successor and addition remain total, multiplication is not a total function, and multiplication facts are expressed through a relation $mult(x, y, z)$. Later formulations package finite systems as $IS\#_D(\beta)$, where β is a finite list of proper axioms and D is the selected deduction apparatus [6].

4 The Proflog SJAS Profile

The implemented profile constructs a finite $IS\#_D(\beta)$-style system whose D is Proflog's ordinary first-order/equality tableau kernel. The builder accepts a profile choice, a finite beta list, reflected Proflog clauses, and external Proflog clauses. It then generates:

- Group-Zero and Group-1 records for the base language and U-grounding prelude;
- Group-2 records for user beta axioms;
- Group-2b records for reflected user clauses;
- a generated Group-3 self-consistency formula;
- object-language `axiom-member` facts over the active finite system code.

Constructing the system does not itself run proof search. Evaluation is query-triggered. A direct Proflog query calls the compiled program's Procedure Call Rule. An SJAS theorem query proves from the generated axiom basis. This distinction is central to the authoring model. A reflected clause such as

```
demo(x) :- x = 1.
```

becomes both executable code and a Group-2b axiom citeable by the internal proof predicate. An external clause remains ordinary executable Proflog code but does not change the system code or generated Group-3 formula.

5 Arithmetized Syntax and Proof Codes

The first implementation used finite labels for formulas and proof targets. That was useful as a bootstrap, but it did not constitute arithmetized syntax. The current profile instead assigns formulas, systems, and proof certificates byte/base-64 codes. In the compact representation these appear as first-order terms of the shape

```
(code-N b0 ... bN-1)
```

where each byte is a small SJAS binary numeral. A stronger U-grounding code format writes public codes as ordinary binary numerals over 0, 1, `dbl`, and `add`. The latter makes the code representation live inside the base language, rather than in generated `code-N` constructors.

The promoted predicates consume these codes. `wff` and formula-class predicates decode formula bytes structurally. `neg-pair` relates a formula code to the code of its negation. `subst-code` performs the fixed-point substitution used by the Level-1 Group-3 sentence. The predicate `tableau-proof/3` decodes the supplied proof code and invokes the Proflog kernel with the decoded proof term supplied as the certificate to be checked. Thus proof validity is not decided by a host-side lookup table.

The source boundary still constructs finite code terms from Proflog formulas before a query begins. After construction, however, the checked predicates receive object-language codes and route them through SJAS decoding and proof checking. In the U-grounding profile, public formula, system, and proof codes are numerals in the SJAS arithmetic vocabulary, and relation-backed decoding records the fixed-radix multiplication step used to recover byte strings.

6 Examples

The worked examples exercise three relevant modes.

Reflected and external programs. With a reflected `demo` clause, both ordinary Proflog evaluation and SJAS theorem evaluation can prove `demo(1)`. Adding an external application clause lets the compiled program prove its ground call, but leaves the system code and Group-3 code unchanged. This shows that "writing a program in the SJAS" means adding a finite reflected extension before Group-3 generation, not merely calling an SJAS-flavored library from outside.

Beta axiom versus executable procedure. The system can place

```
forall x. mult(2, 2, x) -> composite(x)
```

in beta. Then `composite(4)` is an SJAS theorem, but there is no runtime `composite/1` procedure. Alternatively, a reflected procedure

```
composite(x) :- mult(2, 2, x).
```

is executable through the Procedure Call Rule and is also represented as a Group-2b axiom. The executable version supports answer mode: querying `composite(x)` returns the binary numeral for 4.

Certificate checking. For a generated beta axiom, the profile can validate a proof certificate for that theorem code. The same certificate is rejected for an unrelated theorem code. A formal `sjas-axiom` certificate succeeds only when `axiom-member(system,theorem)` is true for the active system. An inconsistent control system containing `false` validates a contradiction certificate, demonstrating that the contradiction code is a substantive proof target rather than a placeholder.

7 Evaluation

The focused SJAS regression suite was rerun on 18 May 2026 against commit `14d3150`. It records:

```
lein test-proflog-sjas
Ran 35 tests containing 221 assertions.
0 failures, 0 errors.
```

The explicit slow selector records:

```
lein test-proflog-sjas-slow
Ran 5 tests containing 22 assertions.
0 failures, 0 errors.
```

The tests cover language shape, formula classes, reflected and external authoring boundaries, U-grounding arithmetic, forward proof, answer mode, partial synthesis, structural formula decoding, proof-code decoding, the core SJAS proof predicates, and the generated Level-1 fixed-point path.

Several tests are intentionally semantic rather than merely structural. For example, `tableau-proof/3` accepts a real encoded certificate for a beta axiom and rejects the same certificate when paired with an unrelated theorem code. `subst-code/2` computes the Level-1 fixed-point substitution and rejects the system code as the wrong substitution source. The U-grounding tests assert that generated systems expose no `code-N` constructors in their language signatures and that bound-code decoding cites the byte-cons and radix-shift proof steps.

8 Limitations

The current implementation is a finite ordinary-tableau $IS\#_D(\beta)$ substrate. It does not implement Tab-1 or proof-list theorem reuse. Open proof-code synthesis is also operationally expensive. The consistency of the generated system remains a mathematical metatheorem in Willard's sense; the implementation demonstrates executable object-language reflection predicates and generated self-consistency axioms, not a machine-checked proof of the metatheorem.

The programming-language question remains sharper than the current prototype: the absence of total multiplication must have observable consequences. The implementation already removes `mul/2` from the SJAS language and routes proof predicates through inspectable U-grounding codes. Future work should push more syntax construction and proof-code manipulation fully into the object-language arithmetic so that the cost and limitations of replacing total multiplication by `mult/3` are directly experienced by programs.

The U-grounding proof predicates also have a pragmatic ground-entry path for already-ground public code numerals. That path destructures the code term one byte at a time to avoid stack overflow in the miniKanren host, then delegates to the same structural byte and formula/proof decoders. It does not synthesize open codes and is not an additional SJAS theorem rule.

9 Related Work

The mathematical source for this implementation is Willard's work on self-verifying and self-justifying axiom systems, especially the ordinary tableau line in which total multiplication is replaced by a multiplication graph relation [3–6]. Those papers establish the consistency phenomenon and the shape of finite $IS\#_D(\beta)$-style systems. Proflog contributes a different perspective: it asks what it takes for such a system to become an executable programming substrate with visible authoring and proof-checking boundaries.

The implementation is also closely related to Fitting's original Proflog proposal [1]. Fitting used semantic tableaux as a logic-programming operational semantics, while the present system reuses the same kind of proof-search machinery as the deduction method whose certificates are coded and checked. Clojure's `core.logic` provides the relational implementation substrate [2]; it is not the object theory being justified, but it supplies the host relations, answer mode, and search control needed to expose the SJAS profile as executable Proflog code.

10 Conclusion

Proflog provides a natural executable setting for Willard-style self-justifying axiom systems because its programs are already proof-search objects. The SJAS profile turns that observation into a concrete system: finite reflected axiom bases, generated self-consistency formulas, arithmetized formula and proof codes, and certificate checking through the ordinary tableau kernel. This makes SJAS not only a metamathematical construction but also a declarative-programming experiment.

References

1. Fitting, M.: Tableaux for logic programming. Journal of Automated Reasoning **13**(2), 175–188 (1994), `https://melvinfitting.org/bookspapers/pdf/papers/LPTableaus.pdf`
2. Nolen, D., contributors: core.logic. `https://github.com/clojure/core.logic` (2026), clojure logic programming library. Accessed 2026-05-18
3. Willard, D.E.: Self-verifying axiom systems, the incompleteness theorem and related reflection principles. Journal of Symbolic Logic **66**(2), 536–596 (2001)
4. Willard, D.E.: How to extend the semantic tableaux and cut-free versions of the second incompleteness theorem almost to robinson's arithmetic q. Journal of Symbolic Logic **67**(1), 465–496 (2002)
5. Willard, D.E.: A detailed examination of methods for unifying, simplifying and extending several results about self-justifying logics (2011), `https://arxiv.org/abs/1108.6330`
6. Willard, D.E.: On the significance of self-justifying axiom systems from the perspective of analytic tableaux (2013), `https://arxiv.org/abs/1307.0150`